Q1. Direction vs Magnitude

- Same cosine similarity → **same direction (angle)** relative to query
- Different Euclidean distance → **different magnitude (length)**

👉 **Conclusion:**

- Cosine similarity → captures **direction**
- Euclidean distance → captures **magnitude + direction**

Q2. ef_search & F1 Score

- ef_search controls how many candidates are explored in HNSW
- Increasing it:
  - Improves **recall**
  - May reduce **precision**

👉 Accuracy alone is misleading.
👉 F1 score balances:

- Precision (relevant retrieved)
- Recall (coverage of relevant items)

✓ So F1 is important for **balanced evaluation**

Q3. Retriever that fixes poor embeddings

👉 **Hybrid Retriever (Dense + Sparse)**

- Dense → semantic meaning
- Sparse (BM25) → keyword matching

✓ Combines both → reduces errors due to weak embeddings

Q4. What to update after embedding model change?

You **must re-index everything**

✓ Update:

- Stored document embeddings
- Vector index (FAISS / HNSW / etc.)

👉 Otherwise:

- Query embeddings ≠ stored embeddings → bad retrieval

Q5. High-dimensional metric choice

Given:

- 500K vectors
- 256 dimensions
- 100 QPS

👉 Best metric: **Cosine Similarity**

Why:

- High dimensions → Euclidean distances become less meaningful (curse of dimensionality)
- Cosine focuses on **angle (semantic similarity)**

✓ Computational advantage:

- Can normalize vectors → faster dot product
- More stable behavior in high dimensions

☑ Section B

Q1 (a) Limitation of scoring

👉 Limitation:

- Ignores **term importance (weights)**
- Treats all terms equally
- Doesn't reflect query intent

Q1 (b) Weighted scoring

Given:

Term	D1	D2	D3
vector	2	1	0
retrieval	3	1	2
search	1	3	4

Weights:

- vector = 0.2
- retrieval = 0.5
- search = 0.3

Compute scores:

D1:

= (2×0.2) + (3×0.5) + (1×0.3)
= 0.4 + 1.5 + 0.3 = **2.2**

D2:

= (1×0.2) + (1×0.5) + (3×0.3)
= 0.2 + 0.5 + 0.9 = **1.6**

D3:

= (0×0.2) + (2×0.5) + (4×0.3)
= 0 + 1 + 1.2 = **2.2**

Answer:

✓ Why lower frequency doc can rank higher:

- Because weights matter more than raw frequency
- Important terms (like “retrieval”) dominate score

✓ Dominating term:
👉 **retrieval (weight 0.5)**

Q2 (a)

1. Why search may stop early:

- Approximate search (like HNSW)
- Limited exploration (low ef_search)
- Local optimum reached early

---

2. How to improve:

- Increase **ef_search**
- Explore more nodes
- Tradeoff: slower but more accurate

Q2 (b) Cosine Similarity

Given:

Q = (2, -1, 2, 1)

Step 1: Formula

Cosine similarity:

$$\frac{Q \cdot D}{|Q||D|}$$

Step 2: Compute magnitudes

Vector Magnitude	
Q	√(4+1+4+1) = √10
D1	√(16+4+16+4) = √40
D2	√(4+0+1+9) = √14
D3	√(1+1+4+0) = √6

Step 3: Dot products

Q · D1:

= (2×4) + (-1×-2) + (2×4) + (1×2)
= 8 + 2 + 8 + 2 = **20**

Q · D2:

= (2×2) + (-1×0) + (2×1) + (1×3)
= 4 + 0 + 2 + 3 = **9**

Q · D3:

= (2×1) + (-1×-1) + (2×2) + (1×0)
= 2 + 1 + 4 + 0 = **7**

Step 4: Final cosine values

D1:

= 20 / (√10 × √40)
= 20 / √400 = 20/20 = **1.0**

D2:

= 9 / (√10 × √14) ≈ **0.76**

D3:

= 7 / (√10 × √6) ≈ **0.90**

Final Ranking (Cosine):

👉 **D1 > D3 > D2**

Why D1 dominates in one metric but not another?

✓ In cosine:

- D1 is exact scaled version of Q → perfect similarity

✓ In Euclidean:

- Magnitude differences matter
- A closer vector in space (like D3) might rank better

🔚 **Final Takeaway**

- Cosine → direction (semantic similarity)

- Euclidean → magnitude + distance
- Weighted scoring → **importance beats frequency**
- HNSW tuning → accuracy vs speed tradeoff